LAPORAN PRAKTIKUM

TEKNOLOGI IOT 2



Disusun Oleh :

Muhamad Ramadhani (25782045)

Dosen Pembimbing :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T..

Agus Ambarwari, S.Pd., M.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

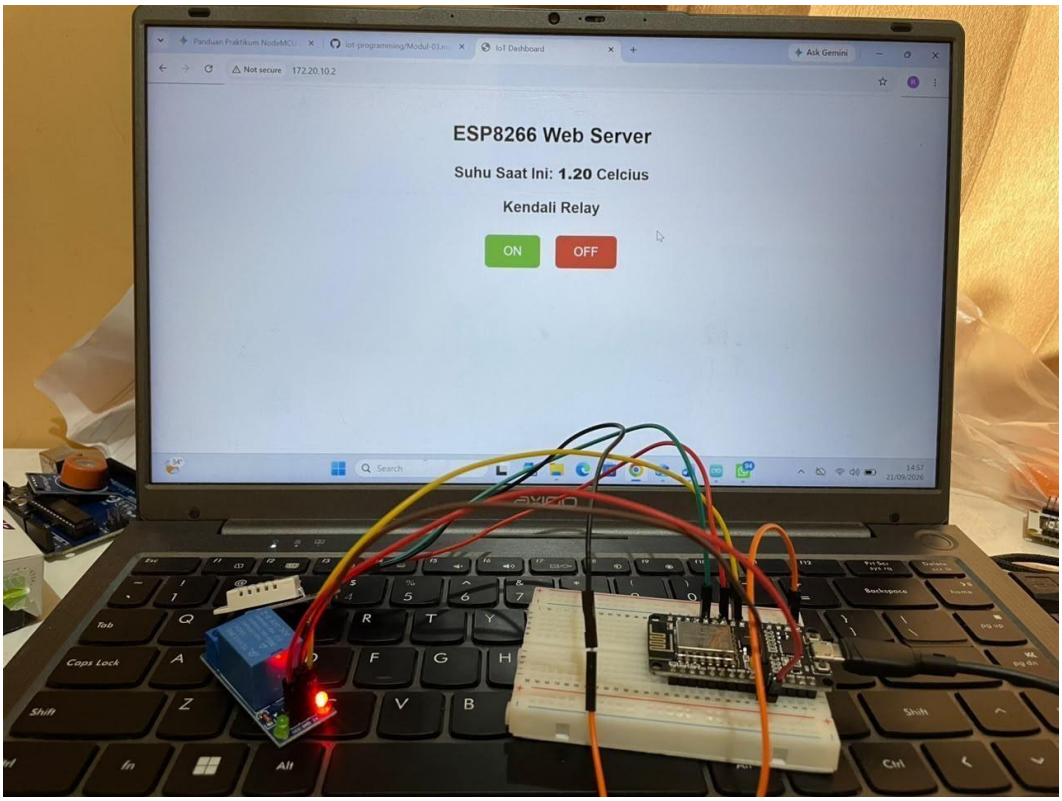
3.7 Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur

Antarmuka (*User Interface*) IoT tidak boleh statis. Saat ini, *Web Server* kita tidak mampu memperbarui informasi suhunya secara otomatis ke layar pengunjung tanpa diperintah. Kita harus menyelesaikannya.

Skenario Proyek (*Web Dashboard*): Anda akan bertindak sebagai *Full-Stack IoT Developer* untuk memutakhirkan kapabilitas *Web Server* pada **Latihan 2**, sembari mengevaluasi limitasi arsitektur tradisional HTTP.

- **Tujuan:** Mampu memanipulasi struktur kerangka HTML mentah yang tersimpan di memori NodeMCU untuk mencapai interaktivitas tampilan otomatis (*Auto-Refresh*). • **Tingkat Kesulitan:** Menengah
- **Instruksi:**
  i. Lakukan modifikasi kode hanya pada sisi **NodeMCU Server (Latihan 2)**.
  ii. Modifikasi variabel Raw Literal index_html. Sisipkan perintah *Meta Refresh HTML* (sebuah Tag standar web kuno namun ampuh) persis di dalam cakupan *tag* <head> untuk memaksa *browser* memuat ulang halaman secara sekuensial setiap 5 detik.
  iii. *Hint (Petunjuk):* Sintaks standar yang harus disisipkan adalah <meta http-equiv="refresh" content="5">. iv. Setelah selesai, buka *browser*, dan amati layarnya. Biarkan selama 20 detik tanpa menyentuh *mouse/keyboard*.
- **Instruksi Analisis Arsitektur:**
  i. Setelah Auto-Refresh berjalan, cobalah tekan tombol *ON / OFF* (kendali LED) dari *browser*. ii. Apa kelemahan visual/interaksi (*flickering*) yang Anda alami saat menggunakan metode *Meta Refresh* HTTP konvensional seperti ini untuk sistem yang mengklaim diri sebagai *RealTime Dashboard*? Tulis evaluasi logis Anda di Laporan Praktikum (Anda bisa membahas masalah *bandwidth*, layar berkedip putih, atau potensi bentrokan jika Anda mengeklik tepat di detik ke-5).
- **Kriteria Keberhasilan:** ○ Nilai suhu dan kelembapan di *browser* otomatis berganti seiring kondisi lingkungan tanpa perlu mengeklik tombol *refresh* di *browser*.
  ○ Jawaban analisis mengenai kelemahan protokol HTTP GET statis tertuang jelas di dokumentasi tugas.

HASIL DARI TUGAS



1. Implementasi Modifikasi Untuk mencapai interaktivitas tampilan otomatis, kerangka HTML pada NodeMCU Server telah dimodifikasi dengan menyisipkan *Tag Meta Refresh* pada bagian <head>. Sintaks yang ditambahkan adalah <meta http-equiv="refresh" content="5">, yang memaksa *browser* untuk melakukan *HTTP GET Request* secara **otomatis setiap 5 detik.** Hasilnya, nilai pemantauan suhu dapat diperbarui tanpa intervensi manual dari pengguna.

2. Analisis Arsitektur dan Kelemahan HTTP Tradisional Meskipun modifikasi di atas berhasil menciptakan ilusi dasbor *real-time*, pendekatan menggunakan *Meta Refresh* HTTP konvensional ini memiliki batasan arsitektur yang fatal jika diterapkan pada sistem IoT industri. Berdasarkan pengujian yang dilakukan, terdapat tiga kelemahan utama:

- **Kelemahan Visual (Screen Flickering):** Setiap siklus 5 detik, layar *browser* akan berkedip putih sesaat (*flickering*). Hal ini terjadi karena *browser* harus menghancurkan (*destroy*) seluruh tampilan (DOM HTML) dan menggambarnya ulang dari nol. Tampilan visual yang tidak mulus ini menghasilkan *User Experience* (UX) yang buruk.
- **Kendala Interaksi (Collision / Bentrokan):** Terdapat potensi "tabrakan" (*collision*) antara aksi pengguna dan jadwal muat ulang halaman. Jika administrator menekan tombol kendali Relay (ON/OFF) tepat pada detik ke-5 saat halaman sedang melakukan *refresh*, maka perintah klik tersebut berisiko terabaikan (*dropped*) atau halaman menjadi tidak responsif.
- **Inefisiensi Bandwidth dan Beban Prosesor (Overhead):** Protokol HTTP bersifat *stateless*. Saat halaman dimuat ulang, NodeMCU harus merakit dan mengirimkan ulang keseluruhan struktur HTML, CSS, dan teks ke *browser*. Padahal, data aktual yang berubah hanyalah beberapa *byte* (nilai suhu). Pengiriman data statis secara berulang-ulang ini merupakan pemborosan *bandwidth* jaringan dan membebani kinerja memori/prosesor ESP8266 secara tidak efisien.

3. Kesimpulan Penggunaan protokol HTTP statis dengan metode *Meta Refresh* sangat tidak direkomendasikan untuk *Real-Time IoT Dashboard* berskala besar. Sebagai solusi di masa depan, arsitektur antarmuka harus ditingkatkan menggunakan teknologi **AJAX (Asynchronous JavaScript and XML)** atau **WebSockets**. Dengan teknologi tersebut, sistem dapat mengambil data suhu di latar belakang dan mengubah angka di layar secara spesifik tanpa perlu memuat ulang keseluruhan halaman web.

4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*

Sinyal Digital (*ON/OFF*) sangat berguna, namun bagaimana jika Anda ditugaskan merancang antarmuka penyesuai intensitas redup-terang lampu (*Dimmer*) dari *smartphone*?

Skenario Proyek (*Smart Lighting*): Anda akan mengeksplorasi penggunaan modulasi sinyal listrik analogWrite() (atau dikenal dengan fitur PWM) dan memadukannya dengan teknologi masukan elemen *Slider / Range* dari HTML5.

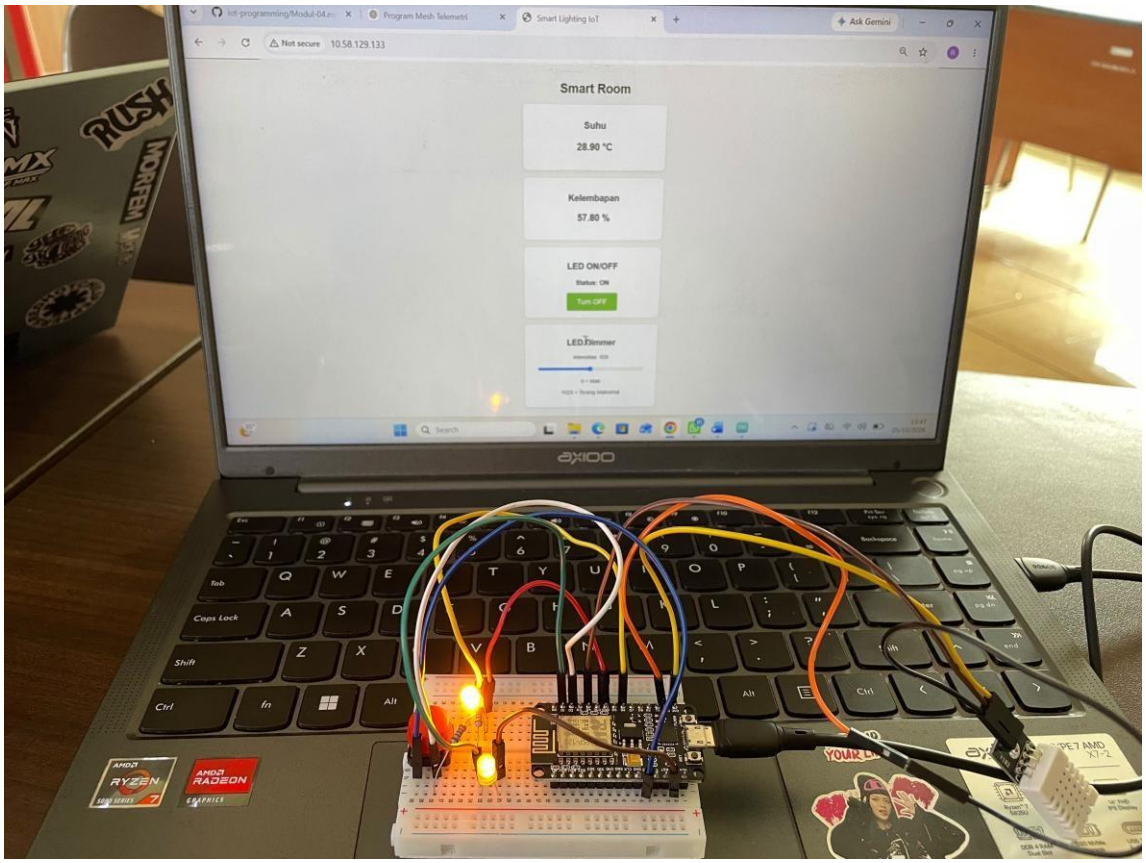
- **Tujuan:** Menguasai transmisi paket *String* WebSocket berkelanjutan (*continuous packet*) dari manipulasi antarmuka layar *slider* untuk merombak parameter persentase daya komponen aktuator. • **Tingkat Kesulitan:** Lanjut
- **Instruksi Perangkaian:**
  i. Tambahkan sebuah komponen **LED** yang dilengkapi **Resistor 220 Ohm** (sebagai pengaman arus).
  ii. Hubungkan kaki Anoda (positif) sistem LED tersebut ke pin **D1 (GPIO 5)**.
- **Instruksi Pemrograman (Front-End & Back-End):**
  i. Modifikasi desain kerangka web HTML Anda. Sisipkan elemen input *Slider* HTML. *(Hint sintaksis: <input type="range" min="0" max="1023" id="pwmSlider" onchange="sendPWM(this.value)">)* ii. Buat sebuah fungsi JavaScript bernama sendPWM(value) yang akan mengirimkan paket *string* teks berformat khusus (misalnya "pwm,512") ke NodeMCU melalui *websocket*.
  iii. Pada ruang fungsi C++ handleWebSocketMessage() di sisi server (NodeMCU), pecah (*parsing*) atau periksa string tersebut menggunakan fitur deteksi *substring* atau bahasa pemrosesan C++ lainnya. iv. Jika Server NodeMCU sukses mendeteksi dan mengambil angka murninya, panggil fungsi aktuator analogWrite(pinLED, nilaiPWM) untuk secara elegan meredupkan atau menerangkan LED LED fisik secara seketika sesuai gesekan layar Anda!
- **Kriteria Keberhasilan:** ○ Di layar gawai Anda, hadir tuas gulir horizontal (*Slider*). ○ LED di dunia nyata dapat diredupkan dan diterangkan secara bertahap saat Anda menggeser tuas *slider*, bebas dari lag (*real-time response*).
  - Kendali LED dan pembacaan Sensor *DHT* bawaan Praktikum 1 dipastikan masih bekerja berdampingan dengan sempurna.

[PENTING!]

Instruksi Pengumpulan Tugas Praktikum Seluruh pengerjaan praktikum dan tugas (meliputi *source code* berformat .ino, dokumentasi foto/video sirkuit yang berhasil, tangkapan layar *Serial Monitor*, serta laporan praktikum tertulis .PDF) harus dikumpulkan dengan cara melakukan **commit dan push ke repositori GitHub pribadi Anda masing-masing**.

Tautkan/kumpulkan *URL repositori GitHub* Anda pada sistem manajemen pembelajaran (LMS) kampus sebagai bukti penyelesaian praktikum ini.

Hasil dari tugas



Disini menunjukan hasil jika instensitas semakin tinggi maka lampu led akan semakin terang menyala dan begitu juga sebaliknya jika intensitass rendah maka lampunyya juga akan semakin redup

